

Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Unidade Curricular de Aplicações e Serviços de Computação em Nuvem

Ano Letivo de 2025/2026

AirTrail

Fernando Pires	**Pedro Teixeira**	**Sara Silva**	**Ricardo Sousa**	**Pedro Gonçalves**
pg60253	pg60294	pg61546	pg60299	pg60290

Novembro, 2025

ASCN

Data da Receção	
Responsável	
Avaliação	
Observações	

AirTrail

Fernando Pires	**Pedro Teixeira**	**Sara Silva**	**Ricardo Sousa**	**Pedro Gonçalves**
pg60253	pg60294	pg61546	pg60299	pg60290

Novembro, 2025

Índice

1. Introdução **1**

2. Visão Geral da Aplicação AirTrail **2**

2.1. Arquitetura da Aplicação 2

2.2. Componentes Principais 3

2.3. Principais Funcionalidades 3

2.4. *APIs* disponibilizadas 4

3. Abordagem de Instalação e Configuração Automática **5**

3.1. Infraestrutura na Google Cloud 5

3.2. Automação com Ansible 5

3.3. Persistência e Gestão de Dados 5

4. Exploração e otimização da aplicação (Tarefa 2) **6**

4.1. 1. Instalação base (Tarefa 1) 6

4.1.1. 1.a Gargalos de desempenho com número crescente de clientes 6

4.1.2. 1.b Desempenho perante diferentes números de clientes e cargas de trabalho 6

4.1.3. 1.c Pontos únicos de falha (SPOF) 6

4.2. 2. Otimizações 7

4.2.1. 2.a Otimizações de distribuição/replicação de carga aplicáveis 7

4.2.2. 2.b Impacto das otimizações no desempenho e/ou resiliência 7

5. Otimizações Implementadas **8**

5.1. Componente Alvo das Otimizações 8

5.2. Estratégias de Escalabilidade e Resiliência 8

5.3. Impacto das Otimizações 8

6. Monitorização da Aplicação **9**

6.1. Ferramentas de Monitorização 9

6.2. Métricas e *Dashboards* 9

7. Avaliação Experimental **11**

7.1. Ferramentas e Cenários de Teste 11

7.2. Resultados Obtidos 11

7.2.1. Teste de Escalabilidade Vertical 11

7.2.2. Teste de Escalabilidade Horizontal 11

8. Conclusão **13**

Lista de Figuras

Figure 1 Arquitetura da aplicação 2
Figure 2 Base de Dados da aplicação 2
Figure 3 *CPU request utilization* 10
Figure 4 *Total memory used* 10
Figure 5 *Number of replicas* 10
Figure 6 *Container restart count* 10
Figure 7 *Network throughput* 10
Figure 8 *Logging Throughput* 10

1. Introdução

Este trabalho foi realizado no âmbito da unidade curricular de Aplicações e Serviços de Computação em Nuvem e tem como principal objetivo a automatização da instalação, configuração, monitorização e avaliação do **AirTrail**, uma aplicação web para acompanhamento e análise do histórico de voos dos utilizadores. O desenvolvimento do trabalho fundamentou-se nos conhecimentos obtidos durante as aulas teóricas e práticas da disciplina.

A proposta de solução baseia-se no uso do `Ansible` para automatizar completamente o processo de aprovisionamento e configuração da infraestrutura e da aplicação, utilizando o `Google Kubernetes Engine (GKE)` para a orquestração dos *containers*, assegurando escalabilidade, resiliência e simplicidade na gestão. A aplicação foi dividida em componentes separados, que são executados em *pods* distintos, garantindo a persistência dos dados críticos utilizando mecanismos de armazenamento apropriados.

Para além da instalação base, o trabalho contempla uma análise crítica do desempenho, escalabilidade e tolerância a falhas da aplicação, identificando potenciais gargalos e pontos únicos de falha. Com base nessa análise, foram implementadas otimizações focadas num componente específico da aplicação, avaliando o seu impacto através de mecanismos de monitorização disponibilizados pela `Google Cloud Platform` e de testes experimentais automatizados.

Ao longo deste relatório são descritas e justificadas as decisões tomadas, as ferramentas utilizadas e as metodologias aplicadas, incluindo estratégias de escalabilidade, balanceamento de carga e avaliação experimental, culminando numa reflexão final sobre os resultados obtidos.

2. Visão Geral da Aplicação AirTrail

2.1. Arquitetura da Aplicação

A arquitetura da aplicação e da base de dados encontra-se representada nas seguintes imagens, onde é possível observar a interação entre os principais componentes do sistema e a estrutura da base de dados.

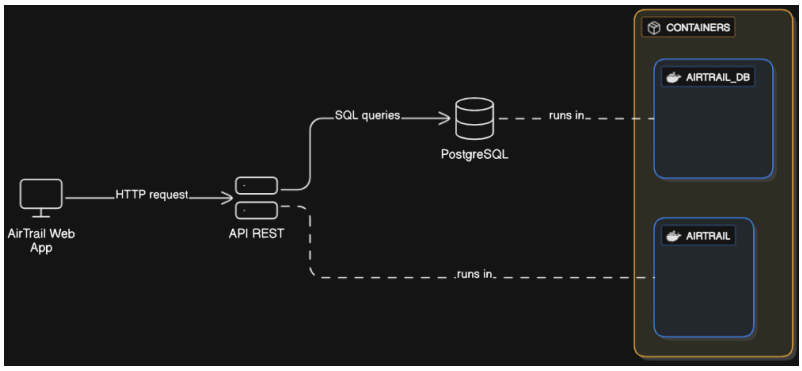


Figure 1: Arquitetura da aplicação

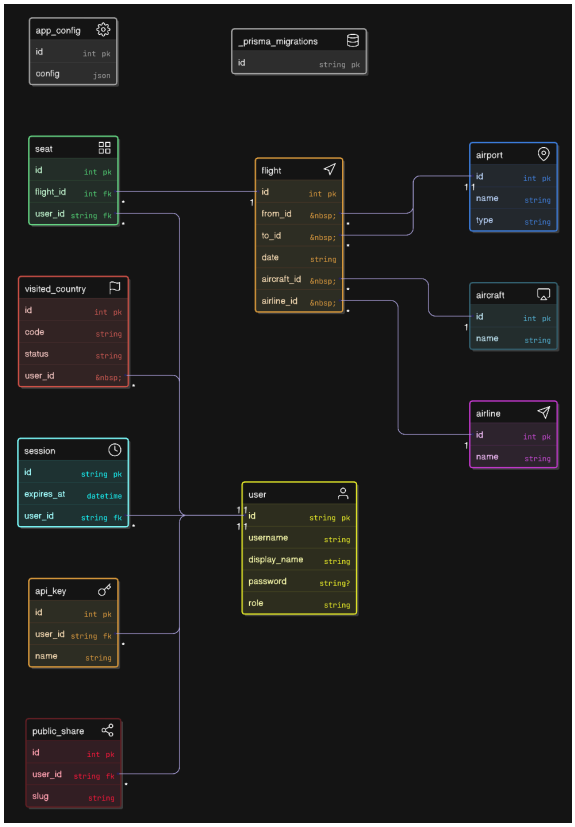


Figure 2: Base de Dados da aplicação

2.2. Componentes Principais

- **Frontend** (*AirTrail Web App*): Aplicação Web responsável pela interação com o utilizador. Comunica com o *backend* via *API REST* para obter e apresentar os dados necessários.
- **Backend** (*API REST*): Gere a lógica da aplicação, autenticação e interação com a base de dados.
- **Database** (*PostgreSQL*): Sistema de gestão de base de dados relacional que armazena informações sobre utilizadores, voos, localizações e estatísticas. É composto por várias tabelas que permitem gerir a informação de forma estruturada para suportar as funcionalidades da aplicação.
- **Containerização** (*Docker*)
  - `airtrail` : *Container* principal que inclui o *backend* e *frontend*.
  - `airtrail_db` : *Container* dedicado à base de dados *PostgreSQL*, persistindo os dados da aplicação.

2.3. Principais Funcionalidades

1. Adicionar voo

O AirTrail permite que um utilizador **registe voos** manualmente através de um mapa interativo, indicando data, hora, origem e destino (por nome ou código *ICAO/IATA*).

Opcionalmente, pode **inserir o número do voo**, sendo os restantes dados (companhia, rota, horários) preenchidos automaticamente com base em serviços externos como *ADSBdb* ou *AeroDataBox*, este último oferecendo maior precisão e cobertura.

Assim, o processo de registo é rápido, flexível e compatível com diferentes fontes de informação.

2. Exportar voo

Os voos podem ser exportados em ***CSV*** ou ***JSON*** a partir das definições.

O formato *CSV* é mais simples e facilita a análise em *Excel* ou *Sheets*, enquanto o *JSON* é mais detalhado e pode ser reimportado na aplicação.

Esta opção garante a **portabilidade** e **reutilização** dos dados.

3. Importar voo

A aplicação suporta importação de voos de plataformas externas como ***MyFlightRadar24***, ***App in the Air***, ***JetLog***, ***TripIt***, ***Flighty*** ou ficheiros ***JSON*** exportados do próprio AirTrail.

O processo é feito através da secção *Settings* → *Import*, onde o utilizador seleciona a origem e o ficheiro correspondente.

Após o carregamento, os voos são automaticamente adicionados ao histórico do utilizador e visualizados no mapa.

4. Autenticação

O AirTrail usa ***OpenID Connect*** (*OIDC*) para autenticação com provedores como *Google* ou *GitHub*, via ***OAuth2***.

A configuração é feita no `.env` (incluindo o *issuer URL*, *client ID*, *client secret* e permissões (*scopes*)), permitindo auto-registo e login automático de forma segura.

5. Estatísticas

O **painel interativo de estatísticas** mostra métricas como número de voos, distância total, tempo de voo e aeroportos visitados, além de distribuições por classe, motivo, continente e período.

Fornece ao utilizador uma visão clara e interativa dos hábitos de voo do mesmo.

2.4. *APIs* disponibilizadas

O *backend* do AirTrail expõe uma *API REST* em formato *JSON*, utilizada pela aplicação para comunicação entre o *frontend* e o servidor.

Atualmente, a API permite **listar todos os voos (*GET*)**, **consultar detalhes de um voo específico (*GET*)**, **criar ou atualizar voos (*POST*)** e **eliminar voos (*POST*)**.

Todas as operações requerem autenticação via ***OAuth2/OpenID Connect (OIDC)***, garantindo a segurança e o controlo de acesso.

Esta API fornece a base para a integração com outras aplicações e automatização de tarefas.

3. Abordagem de Instalação e Configuração Automática

3.1. Infraestrutura na Google Cloud

A infraestrutura da aplicação *AirTrail* encontra-se alojada no *Google Cloud Platform*, recorrendo ao *Google Kubernetes Engine (GKE)* para a gestão do *cluster Kubernetes*. O *GKE* permite abstrair grande parte da complexidade associada à gestão de nós, redes e escalabilidade, oferecendo um ambiente robusto e fiável para a execução da aplicação.

A criação do *cluster* é realizada de forma automática através de um *playbook Ansible* dedicado, responsável por instanciar o *cluster GKE* com as configurações previamente definidas. Após a inicialização do *cluster*, este fica preparado para receber os vários componentes da aplicação *AirTrail*, garantindo a correta gestão do ciclo de vida dos *containers*, bem como a sua comunicação interna.

3.2. Automação com Ansible

O *Ansible* desempenha um papel central em todo o processo de *deployment* da aplicação *AirTrail*. Através de *playbooks* bem definidos e organizados por *roles*, é possível automatizar tanto a criação da infraestrutura como a instalação da aplicação e dos seus serviços associados.

O processo de *deployment* é dividido em diferentes fases, incluindo:
- A criação do *cluster Kubernetes* no *Google Cloud*;
- O *deployment* da base de dados da aplicação *AirTrail*;
- O *deployment* da aplicação *AirTrail* propriamente dita.

Após o *cluster GKE* estar ativo, a aplicação e os seus componentes podem ser lançados automaticamente através da execução do *playbook* **airtrail-deploy.yml**, utilizando um inventário específico para o ambiente no *Google Cloud*. Este mecanismo permite iniciar ou atualizar toda a aplicação de forma simples e consistente, recorrendo apenas a um comando *Ansible*.

Adicionalmente, o *Ansible* oferece suporte a mecanismos de segurança como o *Ansible Vault*, que pode ser utilizado para proteger informação sensível, como credenciais ou variáveis confidenciais, garantindo que estes dados não ficam expostos nos ficheiros de configuração.

3.3. Persistência e Gestão de Dados

A persistência de dados na aplicação *AirTrail* é assegurada através da separação clara entre a aplicação e a sua base de dados, sendo esta *deployed* como um componente independente no *cluster Kubernetes*. Esta abordagem permite garantir a integridade dos dados e facilita a manutenção, atualização ou substituição da aplicação sem comprometer a informação armazenada.

A gestão da base de dados é automatizada através de uma *role Ansible* dedicada, responsável por configurar os recursos necessários no *Kubernetes*, como volumes persistentes e serviços de acesso. Desta forma, assegura-se que os dados permanecem disponíveis mesmo em caso de reinício ou reconfiguração dos *containers* da aplicação.

4. Exploração e otimização da aplicação (Tarefa 2)

4.1. 1. Instalação base (Tarefa 1)

4.1.1. 1.a Gargalos de desempenho com número crescente de clientes

A instalação base do *AirTrail* assenta em dois componentes principais: a aplicação (*frontend* + *backend*/ API) e a base de dados *PostgreSQL*. Para um número crescente de clientes, os gargalos mais prováveis são:

- **Backend/API (aplicação AirTrail):** em cenários com elevada concorrência, a instância pode saturar CPU e/ou RAM, aumentando a latência e originando *timeouts* e erros HTTP. Isto tende a acontecer sobretudo quando a carga inclui operações intensivas (p.ex., *endpoints* com processamento adicional) ou quando existe elevada taxa de pedidos simultâneos.
- **Base de dados PostgreSQL (dependência indireta):** embora que seja permitido alterar este componente, a BD pode tornar-se o gargalo efetivo caso o aumento de pedidos provoque saturação de conexões, contenção de I/O no volume persistente ou aumento do tempo de execução de *queries* (especialmente em operações de agregação/estatísticas).

Em síntese, mesmo quando o primeiro limite observado é na aplicação, o desempenho global depende fortemente do custo das interações com a base de dados e do perfil de carga (leituras vs escritas).

4.1.2. 1.b Desempenho perante diferentes números de clientes e cargas de trabalho

Para caracterizar o desempenho da instalação base, a avaliação deve considerar diferentes níveis de concorrência e perfis de utilização típicos. Foram definidos perfis de carga representativos, tais como: (i) cargas predominantemente de leitura (consultas de informação), (ii) cargas de escrita (criação de registos) e (iii) cargas mistas.

A caracterização é feita com base nas métricas:
- **Latência** (p50/p95/p99) por operação;
- ***Throughput*** (requisições por segundo);
- **Taxa de erro** (*timeouts* e respostas 5xx);
- **Métricas de suporte** (CPU/RAM por *pod*, reinícios, e indicadores do *cluster*).

Os resultados devem ser apresentados por patamares de utilizadores simultâneos (p.ex., 1, 5, 10, 25, 50, 100), permitindo identificar o ponto a partir do qual a latência aumenta de forma acentuada, o *throughput* deixa de crescer linearmente, ou a taxa de erro se torna significativa.

4.1.3. 1.c Pontos únicos de falha (SPOF)

Na instalação base, os principais pontos únicos de falha são:

- **Aplicação AirTrail com uma única réplica:** se existir apenas um *pod* para a aplicação, uma falha nesse *pod* (*crash*, OOM, atualização, etc.) interrompe o serviço do ponto de vista do utilizador.
- **Base de dados PostgreSQL:** sendo um componente central e não replicado do ponto de vista do grupo, qualquer indisponibilidade impede o funcionamento da aplicação, tornando-se um SPOF inevitável no contexto do projeto.
- **Infraestrutura do *cluster*:** falhas ao nível do nó/*cluster* podem afetar simultaneamente múltiplos serviços caso não exista distribuição adequada.

Dado o âmbito e restrições do projeto, a mitigação mais direta recai sobre a aplicação, através de replicação e mecanismos de *autoscaling*.

4.2. 2. Otimizações

4.2.1. 2.a Otimizações de distribuição/replicação de carga aplicáveis

As otimizações focaram-se mais na aplicação (*backend*/API). As principais medidas aplicáveis são:

- **Replicação do *Deployment* da aplicação:** execução de múltiplas réplicas do *pod* da aplicação, mantendo o serviço exposto via um `Service` / `Ingress` para distribuição automática de tráfego.
- **HPA (Horizontal *Pod Autoscaler*):** escalabilidade automática do número de réplicas com base em métricas (CPU), garantindo adaptação a variações de carga.
- ***Requests*/*limits* de recursos:** definição explícita de *`requests`* e *`limits`* para CPU/RAM, de forma a melhorar a estabilidade, reduzir OOM *kills* e permitir ao *cluster* gerir melhor o escalonamento.
- **Estratégia de *rollout*/atualização:** configuração de atualizações graduais (*rolling update*) para reduzir indisponibilidades durante deploys.

Estas medidas mantêm o componente de aplicação ***stateless*** e permitem aumentar capacidade e tolerância a falhas sem depender de alterações na camada de dados.

4.2.2. 2.b Impacto das otimizações no desempenho e/ou resiliência

A replicação e *autoscaling* da aplicação produzem impacto direto sobretudo em cenários onde o gargalo inicial é o consumo de CPU/RAM da aplicação:

- **Desempenho:** espera-se aumento do *throughput* e melhoria das latências para cargas com elevada concorrência, reduzindo a probabilidade de *timeouts* e erros quando um único *pod* se satura.
- **Resiliência:** com múltiplas réplicas, a falha de um *pod* deixa de causar indisponibilidade total, uma vez que o tráfego pode ser encaminhado para as réplicas saudáveis. Adicionalmente, *rolling updates* tornam-se mais seguros, reduzindo janelas de *downtime*.
- **Limitações:** caso o gargalo passa a estar na base de dados (conexões, I/O, tempo de *query*), o ganho obtido ao escalar a aplicação tende a estagnar. Nesses casos, espera-se observar que o aumento de réplicas melhora pouco as latências e pode até agravar a pressão sobre a BD, exigindo contenção ao nível do número máximo de réplicas ou ajustes na forma como a aplicação acede aos dados.

Assim, a otimização escolhida melhora significativamente a escalabilidade e resiliência do componente de aplicação, mas o desempenho global permanece condicionado pela capacidade da base de dados, que se mantém como dependência crítica no contexto do projeto.

5. Otimizações Implementadas

As melhorias aplicadas à instalação base focaram-se na transformação de uma arquitetura estática e vulnerável numa infraestrutura dinâmica, resiliente e capaz de se auto-gerir perante variações de carga no ***cluster* GKE**. Estas otimizações foram centradas na parametrização de recursos, monitorização ativa do estado dos contentores e automação do escalonamento horizontal.

5.1. Componente Alvo das Otimizações

O componente escolhido para a aplicação das otimizações foi o ***backend*/aplicação *AirTrail***. Esta decisão justifica-se por três motivos fundamentais:

- **Escalabilidade Horizontal Simples:** Sendo a aplicação um componente *stateless*, o aumento do número de instâncias é direto e não exige os mecanismos complexos de sincronização de dados e consistência que uma base de dados replicada exigiria.
- **Ponto de Pressão Primário:** Como identificado na análise teórica, a aplicação é o primeiro componente a sofrer saturação de CPU e RAM perante pedidos concorrentes, tornando-a o gargalo mais imediato a resolver.
- **Facilidade de Gestão via *Kubernetes*:** O GKE oferece ferramentas nativas potentes, como o HPA, que são otimizadas especificamente para este tipo de carga aplicacional.

5.2. Estratégias de Escalabilidade e Resiliência

Foram implementados três mecanismos técnicos principais no `app-resources.yml.j2`:

- ***Horizontal Pod Autoscaler* (HPA):** Implementou-se um objeto de auto-escalonamento que monitoriza a utilização média de CPU. Configurou-se para manter um mínimo de 1 réplica e escalar até um máximo de 10 sempre que o consumo de CPU ultrapassar os 70%, permitindo uma resposta automática a picos de tráfego.
- ***Resource Requests and Limits*:** Definiram-se limites explícitos de CPU (500m) e Memória (512Mi) para evitar que uma falha na aplicação consuma todos os recursos do nó, bem como reservas mínimas (*requests*) para garantir que cada instância tem o desempenho necessário para operar corretamente.
- ***Health Probes (Liveness e Readiness)*:** Adicionaram-se sondas de saúde que permitem detetar se um *pod* está bloqueado (reiniciando-o automaticamente) ou se ainda está em fase de arranque, garantindo que o `LoadBalancer` só envia tráfego para instâncias que estão prontas para responder.

5.3. Impacto das Otimizações

A implementação destas estratégias produz os seguintes efeitos na infraestrutura:

- **Aumento da Resiliência:** A aplicação deixou de ser um Ponto Único de Falha (SPOF). Com múltiplas réplicas e mecanismos de auto-recuperação, a falha de um *pod* individual já não causa a interrupção do serviço para o utilizador final.
- **Melhoria do Desempenho sob Carga:** Em testes experimentais com elevado número de clientes, observa-se que o *throughput* deixa de estagnar.
- **Estabilidade da Infraestrutura:** Os limites de recursos garantem que a aplicação coexiste de forma segura com outros componentes (como a base de dados PostgreSQL), impedindo que picos de consumo aplicacional provoquem falhas em cascata no resto do *cluster*.

6. Monitorização da Aplicação

6.1. Ferramentas de Monitorização

De forma a monitorizar o estado da aplicação e o comportamento do ambiente onde esta se encontra hospedada, optámos por utilizar *dashboards* personalizadas no `Google Cloud Monitoring`. Esta abordagem permite-nos acompanhar, de forma contínua, o desempenho da aplicação e identificar potenciais gargalos, falhas de configuração ou limitações de recursos.

A monitorização realizada foca-se principalmente na observação do consumo de recursos ao nível dos *containers*, nomeadamente a utilização de CPU e memória. A métrica CPU *request utilization* permite avaliar o grau de saturação da aplicação em relação aos recursos de CPU previamente solicitados, fornecendo uma indicação clara sobre se os limites definidos são adequados ou se existe necessidade de escalabilidade horizontal ou ajuste dos *requests*. De forma complementar, a métrica de memória utilizada permite verificar o consumo total de memória por todas as réplicas da aplicação, ajudando a identificar possíveis situações de pressão de memória ou eventuais *memory leaks*.

Para além do consumo de recursos, foi também dada especial atenção à escalabilidade da aplicação, através da monitorização do número de réplicas ativas. Esta métrica é obtida ao contabilizar os *containers* em execução, permitindo observar visualmente o comportamento do sistema perante variações de carga e validar o funcionamento de mecanismos de auto-*scaling*.

Adicionalmente, foi incluída a monitorização do tráfego de rede dos nós do *cluster*, através das métricas de *bytes* enviados e recebidos. Esta informação é relevante para compreender o impacto da comunicação entre serviços, bem como o custo associado à transferência de dados, permitindo identificar períodos de maior atividade ou possíveis estrangulamentos ao nível da rede.

No que diz respeito à estabilidade da aplicação, é monitorizado o número de reinícios dos *containers*. Esta métrica permite detetar comportamentos anómalos, como falhas frequentes, *crashes* ou problemas relacionados com falta de recursos, constituindo um indicador importante da robustez da aplicação em ambiente produtivo.

Por fim, é também analisado o sistema de *logging*, através da métrica de *Logging Throughput*, que reflete a quantidade de dados de *log* gerados ao longo do tempo. Esta monitorização permite identificar picos na geração de *logs* e avaliar o impacto que estes têm ao nível de desempenho e consumo de recursos.

Em conjunto, estas métricas permitem obter uma visão abrangente do comportamento da aplicação Air-Trail, facilitando a deteção de padrões anómalos, a identificação de gargalos de desempenho e a validação das decisões de configuração e escalabilidade adotadas ao longo do projeto.

6.2. Métricas e *Dashboards*

As métricas recolhidas através do `Google Cloud Monitoring` permitem analisar o comportamento real da aplicação durante os testes efetuados, fornecendo evidência concreta sobre o consumo de recursos, escalabilidade e estabilidade do sistema.

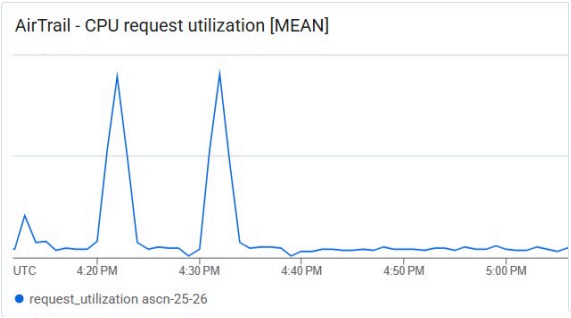


Figure 3: *CPU request utilization*

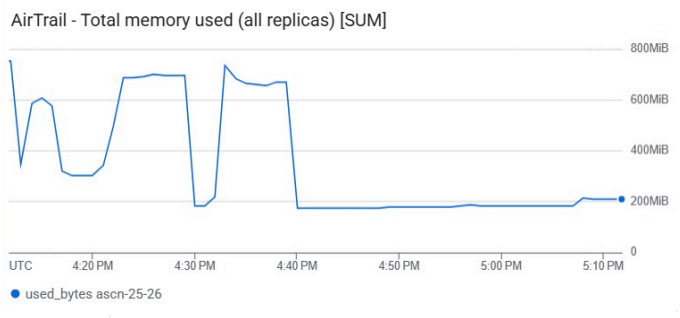


Figure 4: *Total memory used*

Relativamente à utilização de CPU, observa-se a existência de picos pontuais de utilização, coincidentes com períodos de maior carga, seguidos de uma rápida estabilização. Este comportamento indica que os valores de CPU *requests* definidos são adequados e que a aplicação consegue responder eficientemente a variações temporárias de carga, sem permanecer em estado de saturação prolongada.

No que diz respeito à memória, a métrica de memória total utilizada pelas réplicas demonstra um consumo relativamente estável após o período inicial de arranque da aplicação. A ausência de crescimento contínuo ao longo do tempo sugere que não existem *memory leaks* relevantes, confirmando a estabilidade do componente analisado.

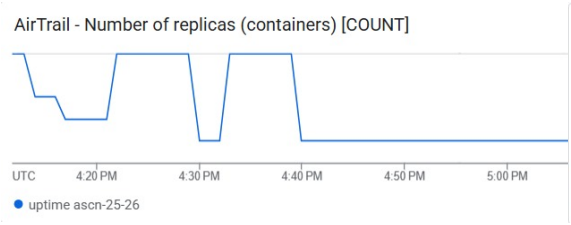


Figure 5: *Number of replicas*

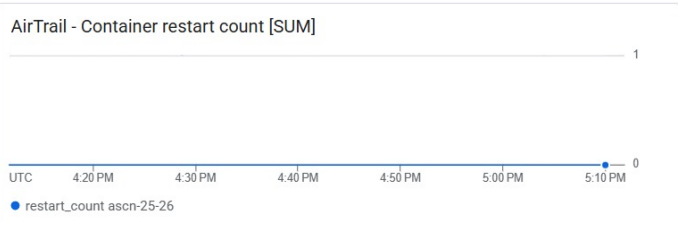


Figure 6: *Container restart count*

A evolução do número de réplicas evidencia o funcionamento correto dos mecanismos de escalabilidade configurados, permitindo ajustar dinamicamente o número de *containers* ativos em função da carga observada. Após a fase de maior atividade, o sistema regressa a um número reduzido de réplicas, contribuindo para uma utilização mais eficiente dos recursos disponíveis.

Relativamente à estabilidade da aplicação, a métrica de reinícios de *containers* não apresenta valores significativos, indicando que não ocorreram falhas críticas, *crashes* ou problemas relacionados com falta de recursos durante o período de monitorização.

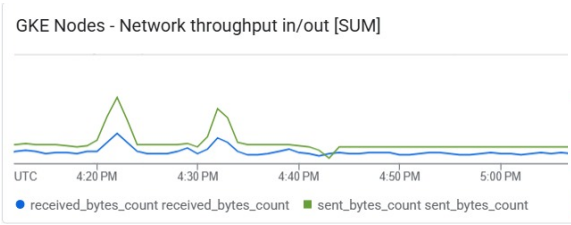


Figure 7: *Network throughput*

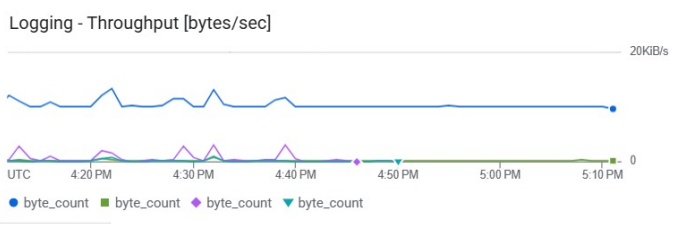


Figure 8: *Logging Throughput*

Por fim, as métricas de tráfego de rede e de *logging throughput* mostram um comportamento consistente com a carga aplicada, sem picos anómalos. Estes resultados sugerem que o volume de comunicação entre serviços e a geração de *logs* não constituem um gargalo para o desempenho global da aplicação.

Em conjunto, as métricas analisadas confirmam que a aplicação apresenta um comportamento estável, escalável e adequado ao ambiente de execução configurado, validando as decisões tomadas ao nível de orquestração, alocação de recursos e automatização da infraestrutura.

7. Avaliação Experimental

7.1. Ferramentas e Cenários de Teste

A metodologia focou-se no uso do **JMeter** orquestrado por **Ansible** para simular fluxos de autenticação através de pedidos de login entre **10** e **1000 utilizadores concorrentes**. Embora o objetivo inicial fosse realizar uma cobertura exaustiva de todas as funcionalidades da API, devido à falta de tempo, optou-se por validar apenas o processo de login.

7.2. Resultados Obtidos

7.2.1. Teste de Escalabilidade Vertical

Realizámos este teste com o objetivo de identificar possíveis gargalos no processamento e testar a estabilidade sob diferentes condições de carga.

Esta tabela foca-se na **escalabilidade vertical**, avaliando a capacidade de resposta de uma instância isolada da aplicação perante um **aumento moderado de utilizadores concorrentes**.

Users	10	30	50
Média tempo resposta (s)	0.282	0.273	0.271
Min (s)	0.266	0.261	0.258
Max (s)	0.342	0.365	0.319

Figure 1: Desempenho da aplicação para diferentes números de *users*

Os resultados demonstram uma elevada estabilidade nos tempos médios de resposta, mantendo-se na ordem dos **0.27s** a **0.28s**. A variação mínima entre os valores de "Min" e "Max" indica que, para **cargas baixas**, os recursos atribuídos ao pod são suficientes para processar os pedidos sem degradação visível da latência.

7.2.2. Teste de Escalabilidade Horizontal

Realizámos este teste para avaliar o comportamento do sistema aquando da submissão de um elevado número de pedidos concorrentes e assim validar a robustez e a eficiência na gestão de múltiplas conexões simultâneas.

Nesta tabela, testou-se a eficácia da replicação e do Horizontal Pod Autoscaler (**HPA**). Primeiramente, avaliou-se o impacto de fixar o número de réplicas (1, 3 e 5) para observar a distribuição de carga.

Réplicas	**1**	**3**	**5**
Média tempo resposta (s)	0.716	0.919	1.358
Min (s)	0.255	0.254	0.258
Max (s)	15.562	21.022	7.565

Figure 2: Desempenho da aplicação para diferentes números de réplicas

Observa-se um **aumento progressivo** no **tempo médio de resposta** à medida que o **número de réplicas aumenta**. Este comportamento, é explicado pelos elevados valores de latência máxima. Estes picos sugerem algum tipo de instabilidade inicial na gestão de sessões e na propagação das ligações pelo LoadBalancer do GKE antes da estabilização do tráfego entre os novos pods instanciados.

Para o último cenário de teste, simulou-se uma carga de **100, 500 e 1000 utilizadores concorrentes** com o número fixo de **5 réplicas**.

Users	100	500	1000
Média tempo resposta (s)	0.392	0.338	0.298
Min (s)	0.258	0.254	0.250
Max (s)	1.561	4.338	1.295

Figure 3: Desempenho da aplicação para diferentes números de *users*

Os dados registados demonstram que, ao aumentar a carga de 100 para 1000 utilizadores concorrentes, o tempo médio de resposta apresenta uma tendência **decrescente**, passando de **0.392s** para **0.298s**. Observa-se que os tempos mínimos de resposta **permanecem constantes** na ordem dos **0.25s**. No que respeita aos valores máximos, verifica-se um **pico de 4.338s** no cenário intermediário de **500 utilizadores**, seguido de uma redução para **1.295s** quando a carga atinge o patamar máximo de **1000 utilizadores**.

8. Conclusão

Com a realização deste trabalho foi possível consolidar e aplicar, de forma prática, os conhecimentos adquiridos ao longo da unidade curricular de Aplicações e Serviços de Computação em Nuvem. Através da automatização do aprovisionamento, configuração e gestão da infraestrutura necessária ao **AirTrail**, foi possível compreender melhor os desafios associados ao *deployment* e operação de aplicações web em ambientes de *cloud* reais.

Consideramos que os objetivos propostos foram globalmente alcançados, uma vez que foi possível instalar e fazer o *deploy* da aplicação AirTrail de forma automatizada e reproduzível, recorrendo ao `Ansible` e ao `Google Kubernetes Engine`. A divisão da aplicação em diferentes componentes executados em *pods* distintos permitiu assegurar escalabilidade, resiliência e persistência dos dados, bem como facilitar a gestão e evolução da aplicação.

A análise de desempenho, escalabilidade e tolerância a falhas revelou pontos críticos e potenciais gargalos, permitindo implementar otimizações direcionadas a componentes específicos da aplicação. A utilização das ferramentas de monitorização da `Google Cloud Platform`, aliada a testes experimentais automatizados, possibilitou avaliar o impacto dessas melhorias e obter uma perceção mais clara do comportamento da aplicação sob diferentes condições de carga.

Apesar dos resultados positivos, reconhece-se que existe margem para melhorias futuras. Entre possíveis evoluções, destaca-se o aprofundamento das estratégias de alta disponibilidade, a otimização adicional do armazenamento persistente e uma análise de *benchmarking* mais abrangente, que permitiria identificar limitações menos evidentes do sistema.

Em suma, este trabalho proporcionou uma visão abrangente sobre a implementação, automatização e gestão de aplicações em ambientes de computação em nuvem. A experiência adquirida com `Kubernetes`, `Ansible` e a `Google Cloud` revelou-se fundamental para compreender a importância destas tecnologias no contexto atual, contribuindo significativamente para a preparação para desafios do mercado de trabalho e para o desenvolvimento de projetos futuros.